

阶段二：后端测试与 Debug 详细工程方案

最快测试计算核心说明

0. 访问网址

<http://ctjs.rearticle.com:28500/docs>

1. 主要步骤

- 点击:【计算引擎】-> [Try it out]
- 输入测试参数, 示例:

```
//单洞测试
{
  "tunnel_type": "single",
  "water_level": "low",
  "K": 0.15,
  "h": 29,
  "p_mm": 1025.2,
  "Kg": 0.00864,
  "K1": 0.000864,
  "K2": 0.00864,
  "cn_condition": "灌溉良好",
  "land_use": "工业用地",
  "grades": 5,
  "r": 7.95,
  "R1": 8.35,
  "R2": 8.57,
  "Rg": 10.57,
  "c": 32,
  "start_chainage": 0,
  "end_chainage": 47,
  "concrete_grade": "C40",
  "rebar_type": "HRB400",
  "Ag": 1091,
  "beta2": 1,
  "Pcrown_crit": 50,
  "P_crit": 500,
  "I_long": 0.02,
  "double_side": true,
  "as_mm": 50,
  "gamma": 10,
  "n_long": 0.012,
  "n_ring": 0.012,
  "I_ring": 0.73,
  "n_lat": 0.012,
  "I_lat": 0.01,
  "S_code_max": 10,
  "S_min": 3,
  "d_ring_default": 0.05,
```

```
"d_long_default": 0.1,
"d_lat_default": 0.08,
"tol_safety_factor": 2,
"aspect_ratio": 0.7
}

//双洞测试
{
  "tunnel_type": "double",
  "water_level": "high",
  "K": 0.15,
  "h": 90.5,
  "ha": 0.0,
  "p_mm": 1025.2,
  "Kg": 0.00864,
  "K1": 0.000864,
  "K2": 0.00864,
  "cn_condition": "灌溉良好",
  "land_use": "居住地",
  "grades": 4,
  "CN": 61.0,
  "r": 7.95,
  "r1": 8.35,
  "r2": 8.57,
  "rg": 9.57,
  "c": 50.0,
  "start_chainage": 0.0,
  "end_chainage": 100.0,
  "concrete_grade": "C40",
  "rebar_type": "HRB400",
  "Ag": 1091.0,
  "D_spacing": 40.0,
  "beta2": 1.0,
  "Pcrown_crit": 100.0,
  "P_crit": 600.0,
  "I_long": 0.02,
  "double_side": true,
  "as_mm": 50.0,
  "gamma": 10.0,
  "n_long": 0.012,
  "n_ring": 0.012,
  "I_ring": 0.73,
  "n_lat": 0.012,
  "I_lat": 0.01,
  "S_code_max": 10.0,
  "S_min": 5.0,
  "d_ring_default": 0.05,
  "d_long_default": 0.1,
  "d_lat_default": 0.08,
  "tol_safety_factor": 2.0,
  "aspect_ratio": 0.7
}
```

- 点击：【Execute】
- 查看计算返回结果【Response body】，进行数据对比

必要说明

- 此后台为临时公网测试，默认关闭，联系我打开(y.wang@reaticle.com, 15696124832)

界面图：

隧道智能排水自适应平台 1.0.0 OAS 3.1

/api/v1/openapi.json

支持单/双洞隧道、高/低水位模型的智能排水计算后端

计算引擎

POST /api/v1/calculate/drainage 执行隧道多维协同智能排水自适应优化计算

排水计算核心接口：
1.接收单洞(38参数)或双洞(41参数)模型数据。
2.自动分发对应的双洞/单洞、高/低水位模型计算。
3.透传包括内力、应力节点在内的所有 Echarts 作图计算结果。

Parameters

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "tunnel_type": "double",
  "water_level": "high",
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://ctjs.reaticle.com:28500/api/v1/calculate/drainage' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "tunnel_type": "double",
    "water_level": "high",
    "L": 0.35,
    "h": 90.5,
    "hs": 0.0,
    "p_m": 1025.2,
    "kg": 0.00064,
    "k3": 0.00064,
    "k2": 0.00064,
    "cn_condition": "隧道良好",
    "land_use": "居住区",
    "grades": 4,
    "CM": 61.0,
    "C": 7.55,
    "r1": 8.35,
    "r2": 8.57,
    "rg": 9.57,
    "C": 50.0,
    "start_chainage": 0.0,
    "end_chainage": 100.0,
    "concrete_grade": "C40",
    "rebar_type": "HRB400",
    "Ag": 1001.0,
  }'
```

Request URL

http://ctjs.reaticle.com:28500/api/v1/calculate/drainage

Server response

Code

Details

200

Response body

```
{
  "tunnel_type": "double",
  "water_level": "high",
  "original_state": {
    "waterhead": 65.77785925960057,
    "safety_factor": 2.2074814654339687,
    "control_M": -380371.63082522154,
    "control_M": -280224.0489299297,
    "control_id": "j",
    "q": 7.274375294660265,
    "Q": 691.0466525027247,
    "p": 657.7785925960057,
    "p_crown": null,
    "p_invert": null,
    "ring_diam_recommend": 0.05,
    "ring_spacing_recommend": 10,
    "long_diam_recommend": 0.1,
    "lateral_diam_recommend": 0.05,
    "lateral_spacing_recommend": 10
  },
  "critical_state": {
    "final_safety_factor": 2.2074814654339687,
    "final_waterhead": 65.77785925960057,
    "p_crit_input": 657.7785925960057,
    "rg_crit": 9.570000000000006,
    "rg_crit": 1.0000000000000005,
    "q": 7.27437529466026,
    "Q": 691.0466525027247,
  }
}
```

Response headers

```
access-control-allow-credentials: true
access-control-allow-origin: http://ctjs.reaticle.com:28500
content-length: 43647
content-type: application/json
date: Sat, 25 Apr 2026 00:55:36 GMT
server: uvicorn
vary: Origin
```

Responses

Code

Description

Links

200

Successful Response

No links

Media type

application/json

Controls Accept header.

Example Value | Schema

```
"string"
```

422

Validation Error

No links

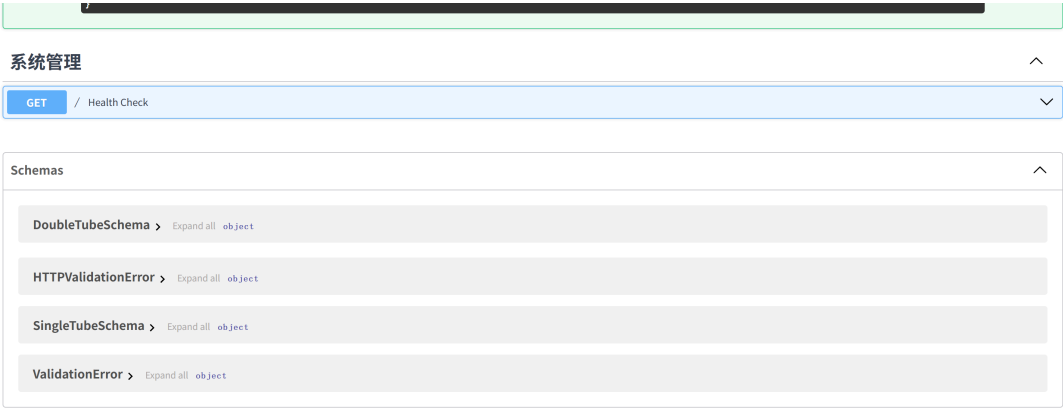
Media type

application/json

Example Value | Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```

5 / 9



Debug 详细工程方案

针对 `uvicorn app.main:app --reload` 启动后的服务化阶段，制定以下具体的工程测试与 Debug 方案，确保 Pydantic 数据契约与 `drainage_engine.py` 计算逻辑的准确对接。

0. 测试python环境启动

在执行 `uvicorn app.main:app --reload` 或进行任何后端 Debug 前，维持环境隔离是前置条件。因为在【阶段一】中，核心依赖包（FastAPI、Uvicorn、Pydantic、Pandas 等）已定点安装在该虚拟环境中。若在未激活状态下运行，系统将默认调用全局 Python 解释器，导致无法找到相关依赖并触发 `ModuleNotFoundError` 报错。

标准操作路径如下：

- 1. 进入项目根目录： 在终端中导航至 `tunnel-drainage-platform\backend`。

特别注意：

- 同步按照 `from app.services import xxx` 的绝对路径格式进行修改

```
# 修改前：
# import double_hige as pdh

# 修改后：
from app.services import double_hige as pdh
```

- 2. 激活虚拟环境：

- Windows 系统：

```
.\venv\Scripts\activate
```

- macOS/Linux 系统：

```
source venv/bin/activate
```

3. **状态验证：** 观察终端命令行提示符，若最前端出现 (venv) 字样，即表明环境隔离已生效。
4. **定位并启动服务：** 根据设定的工程目录架构，入口文件位于 `backend` 目录下。因此需要先进入后端目录再执行启动指令：

```
cd backend
uvicorn main:app --reload
```

1. 基础服务与路由挂载验证

在终端执行 `uvicorn app.main:app --reload` 后，需首先验证服务基座的连通性。

- **进程状态监控：** 检查终端输出日志，确认无模块导入错误（如缺少 `pandas`, `numpy` 等依赖），确认服务监听在 `127.0.0.1:8000`。
- **Swagger UI 挂载确认：** 浏览器访问 `http://127.0.0.1:8000/docs`。检查 OpenAPI 文档是否成功生成，并在界面中定位到目标路由：`POST /api/v1/calculate/drainage`。
- **Schema 渲染检查：** 在 Swagger UI 的 Schema 区域，核对单洞与双洞的请求体结构是否与 `backend/app/models/schemas.py` 定义严格一致，验证默认参数是否已正确显示默认值。

2. API 数据契约与 I/O 测试

利用 Swagger UI 或外部工具（如 Postman）构造 JSON 报文进行接口吞吐测试，验证 Pydantic 模型的拦截与解析能力。

- **单洞标准流程测试：**
 - 构造包含 `tunnel_type: "single"` 及 15 个核心输入、6 个复选参数的 JSON 请求体（忽略 12 个默认参数）。
 - 发送请求，验证 HTTP 状态码返回 200。
 - 检查 Response Body，确认返回体格式包含计算得出的“间距”（如 `spacing: 5.0`）与“孔径”（如 `aperture: 100`）等优化布置结果。
- **双洞标准流程测试：**
 - 构造包含 `tunnel_type: "double"` 及 16 个核心输入参数的 JSON 请求体。
 - 发送请求并验证响应体数据结构，确认第 16 个特有参数被底层算法正确读取。
- **默认值覆写测试：**
 - 在请求体中主动传入 12 个高级设置区参数的修改值。
 - 验证返回结果，确认计算引擎使用了前端下发的新值，而非 `schemas.py` 中的写死保底值。

3. 核心逻辑边界与异常注入 (Debug 重点)

针对 `backend/app/services/drainage_engine.py` 与端点之间的调用链进行异常阻断测试。

- **参数缺失拦截 (422 Unprocessable Entity)：**
 - 故意遗漏 15/16 个核心输入参数中的任意一个（例如不传“隧道总长度”）。
 - 断言 FastAPI 直接返回 422 状态码，且无需进入 `drainage_engine.py`，证明前置路由拦截有效。
- **数据类型校验测试：**
 - 对数值型结构尺寸参数传入字符串（如将半径传入 `"five"`）。

- 确认系统通过 Pydantic 抛出严格的数据类型校验错误。
- **计算引擎断点联调：**
 - 在 `backend/app/api/endpoints/calculate.py` 调用 `drainage_engine.py` 的入口处设置断点。
 - 检查传入核心算法的参数字典 (`kwargs` 或 `dict`) 是否已完成类型转换 (如前端传来的 JSON 字符转为了 Python 原生 `float` 或 `int`)，确保原有原生 Python 排水计算逻辑无需二次转型即可处理。
 - 针对矩阵运算或 Pandas 处理逻辑步进 (Step Over)，监控 `numpy` 数组或 `DataFrame` 在极端工程条件 (如水压值极大) 下的溢出或除零异常。

4. 热更新 (Reload) 与并发验证

- **文件监听测试：** 保持服务运行，修改 `drainage_engine.py` 中的某一计算系数并保存。验证终端触发 `Reloading process` 日志，再次发起相同 POST 请求，核对返回的间距/孔径数值是否已依据新系数发生改变。
- **并发处理排查：** 快速连续发送多次 POST 请求，监控控制台是否有资源互斥锁定报错。确保封装后的 `drainage_engine.py` 函数为无状态设计，不会因全局变量导致不同请求的参数互相污染。

发现的主要bug或问题

python模块与函数名同名歧义

```
from app.services.analyze_tunnel_lining_full import analyze_tunnel_lining_full #精确导入函数本身,避免 from module import * 导入导致的命名空间混乱
from app.services.get_concrete_parameters import get_concrete_parameters
from app.services.get_rock_parameters import get_rock_parameters
```

临时开放公网方法

放行 Uvicorn Host:

原启动命令 `uvicorn app.main:app --reload` 默认绑定 127.0.0.1，仅接收本机请求。需修改启动指令，监听局域网内所有请求：

```
uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

前置关键代码修正：FastAPI CORS 防火墙策略 无论采用上述哪种公网暴露方案，一旦 API 地址从 localhost 变为公网域名，前端页面发起调用时将触发浏览器的跨域资源共享 (CORS) 拦截。在测试阶段，需放宽后端 CORS 限制。

修改 `backend/app/main.py` (或所在入口文件)：

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
```



```
# ... 其他 import ...

app = FastAPI(title="隧道排水计算引擎 API")

# 配置 CORS 中间件
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"], # 测试阶段允许所有域名跨域。生产环境请替换为具体的前端域名
    allow_credentials=True,
    allow_methods=["*"], # 允许所有请求方法 (GET, POST, OPTIONS 等)
    allow_headers=["*"], # 允许所有请求头
)

# ... 路由挂载 app.include_router(...) ...
```

安全建议：测试完成后，及时关闭公网映射；暴露于公网期间，/docs (Swagger UI) 将对所有人可见。若存在敏感业务逻辑，建议在此阶段通过 Nginx 或穿透工具层增加 Basic Auth 认证。

异常诊断：FastAPI JSON 序列化失败

错误 `TypeError: cannot convert dictionary update sequence element #0 to a sequence` 发生在 FastAPI 底层的 `jsonable_encoder` 尝试将 Python 对象转换为 JSON 格式的过程中。

核心原因：底层结构力学计算函数 `analyze_tunnel_lining_full` 返回的结果（即代码中的 `res_original` 和 `res_crit`）内部大概率包含了 NumPy 数组 (`numpy.ndarray`) 或 NumPy 标量 (`numpy.float64`, `numpy.int64`)。FastAPI 原生并不支持直接序列化 NumPy 对象。当它遇到不认识的类型时，会尝试使用 `dict(obj)` 进行兜底转换；如果该对象是一个一维的 NumPy 数组（例如由浮点数组成的数组），`dict()` 会尝试将这些标量视作键值对来迭代，从而触发此报错。

修复方案：引入递归序列化清洗器 需要将底层算法返回的所有带有 NumPy 基因的数据，彻底“降维”转换为 Python 原生的 `list`、`float` 和 `int`，然后再交由 FastAPI 输出。

临界状态的最大迭代次数缺失

```
max_iterations = 500 # 迭代上限，防止死循环
iteration = 0

if now_k <= tol_safety_factor:
    water_head_crit = Hq
    water_head_step = 0.05
    max_head = depth

    while (now_k > tol_safety_factor + 0.001) and (max_head > water_head_crit)
and (iteration < max_iterations):
        water_head_crit += water_head_step
```